IMU filtering on the MRCC flight computer

ICM20948 signal conditioning, attitude estimation and measured results

1. Introduction

The MRCC flight computer carries an ICM20948 as its inertial sensor. It gives the acceleration on three axes, the angular rate on three axes and the magnetic field on three axes as well. The raw output of that sensor can't be used on its own to work out the attitude of the rocket. This is because the accelerometer picks up the vibration of the airframe on top of the gravity, and the gyroscope carries a small offset that never goes away by itself.

If the raw signal is fed straight into the flight logic, the launch detection and the apogee detection will both respond to noise that has nothing to do with what the rocket is actually doing. Because of this, a filter chain was added to the firmware.

The chain runs on every new sample that arrives from the sensor, and it makes a filtered copy of every channel and keeps that copy next to the raw one. Both are written to the SD card on the same line, so one flight gives the before and the after together and there is nothing to line up afterwards. The flight state machine reads the filtered copy. This document covers what each stage does, why it is needed and what the measured effect turned out to be.

2. What is wrong with the raw signal

There are three separate problems in the raw data and they need three different answers. Treating them as one problem is the usual mistake.

The first one is the single sample spike. Every so often one sample jumps by tens of m/s² or by hundreds of deg/s, and then the sample right after it is back to normal. These come from the I2C bus and from the electrical noise of the motor. A low pass filter can't really remove them. It just smears the spike out across the next few samples instead of throwing it away.

The second is the broadband vibration. When the motor is burning the airframe rings, and that ringing lands on the accelerometer as a wide band of noise sitting on top of the real acceleration. This is the thing a low pass filter is actually for.

The third is the gyro zero rate offset, which is also known as the bias. A gyroscope sitting completely still will still report a small rate, and the value is different for every axis and every chip. On the sensor model used here it was around 1.3 deg/s on one of the axes. That sounds small. But the angle is worked out by integrating the rate over time, so a steady 1.3 deg/s turns into about 13 degrees of error after only ten seconds, and it keeps growing from there.

And there is a fourth thing which isn't really noise at all. The accelerometer only points at the gravity when the gravity is the only force acting on it. Under thrust it is measuring

the motor instead of the ground, so the angle worked out from it during the burn is simply wrong. No amount of smoothing will fix a number that is wrong for a physical reason, and this is the part that the low pass filter on its own cannot help with.

3. The filter chain

The chain has five stages and they run in a fixed order on every sample. Table 1 lists them.

Stage	Problem it deals with	How it works
1. Median of three	Single sample spikes	Sorts the last three samples and passes on the middle one
2. Low pass	Broadband vibration	First order filter with the cutoff set in Config.h
3. Zero rate calibration	Gyro bias	Averages the gyro while the rocket sits still on the pad
4. Kalman filter	Gyro drift and accelerometer noise	Two states per axis, being the angle and the gyro bias
5. Trust gate	The accelerometer lying under thrust	Skips the correction step when the total acceleration is not close to 1 g

Table 1 — the five stages of the filter chain, in the order they run.

The order matters quite a lot. If the low pass came before the median, the spike would already be smeared across several samples by the time the median saw it, and then the median wouldn't be able to pick it out any more. In the same way the bias has to be measured before the Kalman filter starts, because otherwise the filter has to spend the early part of the flight learning an offset that could have just been handed to it.

4. Removing the single sample spikes

The first stage keeps the last three samples, sorts them and passes on the middle value. In order for a bad reading to get through, two of the three samples in the window would have to be bad at the same time. A glitch on the I2C bus doesn't behave like that, so in practice the spikes are removed completely rather than being reduced.

The cost is one sample of delay. At the sample rate used here that works out at about 10 ms, which is nothing compared to how fast the rocket actually changes attitude. There is also a small amount of distortion on very fast real edges, and that is accepted on purpose.

5. The low pass filter

The second stage is a first order low pass. The time constant is worked out from the cutoff frequency that is wanted:

$$RC = \frac{1}{2\pi f_c}$$

and then the smoothing factor is worked out from that time constant together with the interval between the two samples:

$$\alpha = \frac{\Delta t}{RC + \Delta t}$$

The filter itself is then just one line, where each new output is the previous output nudged towards the new input:

$$y_n = y_{n-1} + \alpha(x_n - y_{n-1})$$

The detail that matters here is that α is not a fixed number. It is worked out again on every single sample from the Δt that actually happened, which is measured with the microsecond timer. Most simple implementations hard code α once and forget about it. If the main loop then slows down for any reason, the real cutoff frequency quietly moves, and the filter is no longer doing what the setup says it is doing. Working α out each time removes that whole class of problem.

Besides that, the filter is cheap. It is one multiply and one add per channel per sample, so there is no reason to reach for anything heavier unless the measurements say otherwise.

The cutoffs are set in Config.h. The accelerometer is cut at 12 Hz, the gyroscope at 15 Hz and the magnetometer at 3 Hz. The sensor is sampled at roughly 100 Hz, so the highest frequency that can be represented is 50 Hz. Boost and burnout are events that happen over about half a second to a second, which is somewhere around 1 to 2 Hz, so there is a wide gap between what needs to be kept and what needs to be removed. The magnetometer is cut hardest because the magnetic field of the Earth can't change quickly at all, so almost anything fast on that channel is noise.

6. The gyro zero rate calibration

In order to remove the bias it first has to be measured. This is done while the rocket is sitting on the pad, by averaging the gyro over a fixed number of samples:

$$b = \frac{1}{N}\sum \omega_i$$

N is set to 300, which at about 100 Hz works out at roughly three seconds. The measured value is then subtracted from every gyro reading after that.

Two guards are built around it. If any axis reads more than 15 deg/s during the collection, the whole batch is thrown away and the count starts again, so a bump against the rail can't poison the result. And if the rocket never settles, the routine gives up after 20 seconds and leaves the bias at zero rather than storing a value that is known to be wrong. A bias of zero is only a bit worse than the truth, but a bias measured while the board was being moved can be much worse than nothing at all.

The whole thing is non blocking. It collects its samples across normal passes of the main loop, which matters because the design rule for this flight computer is that nothing is ever allowed to halt.

7. The Kalman filter

The gyroscope and the accelerometer fail in opposite ways. The gyro is quiet and smooth over a short window but it drifts, because any small error in the rate gets added up forever. The accelerometer doesn't drift at all, since it always has the gravity to refer back to, but it is noisy and it can be fooled by any other acceleration. The Kalman filter is what lets one cover for the other.

Two states are tracked for each axis, which are the angle itself and the gyro bias. Tracking the bias as a state is what allows the filter to keep removing drift even long after the calibration on the pad was done.

On every sample the angle is pushed forward using the gyro:

$$\theta_k = \theta_{k-1} + \Delta t\,(\omega_k - b_{k-1})$$

Then, if the accelerometer is trusted, a correction is applied. The gain decides how much of the difference between the measurement and the prediction is actually taken:

$$K_0 = \frac{P_{00}}{P_{00} + R}$$

$$\theta_k = \theta_k + K_0\,(\theta_{acc} - \theta_k)$$

The measurement itself comes from the accelerometer, and the two angles can be found from the three axes like this:

$$roll = atan2(a_y, a_z)$$

$$pitch = atan2(-a_x, \sqrt{a_y^2 + a_z^2})$$

R is the number that decides how much the accelerometer is believed. It is set to 0.03 here. If it is made bigger the output gets smoother but also lazier, and it will lag behind a real movement. The two process noise values are set to 0.001 for the angle and 0.003 for the bias, and these control how quickly the filter is allowed to change its mind about each state.

One extra guard is needed on the roll. Roll wraps around at plus and minus 180 degrees, and a wrap looks like an enormous rotation to the filter even though nothing really happened. When the measured angle is more than 90 degrees away from the state, the state is jumped straight to the measurement instead of being corrected towards it.

8. The accelerometer trust gate

This is the stage that makes the rest of it work in an actual rocket rather than only on a bench. The correction step above is only applied when the total measured acceleration is close to one g:

$$| \|a\| - g | < 2.0 \ m/s^2$$

When that test fails, the prediction still runs but the correction is skipped, and the filter coasts on the gyro alone until the accelerometer becomes believable again. It works like a person closing their eyes for a moment when they get dazzled, and then carrying on by memory until they can see properly again.

During the burn the measured acceleration is around eight g, so the test fails and the accelerometer is ignored, which is exactly what should happen. In the test flight the gate was off for 13.0 percent of the total time, covering the burn and part of the coast. This is also the reason the calibration and the bias state matter so much. For those few seconds the gyro is the only thing holding the attitude, so any drift in it goes straight into the answer.

So the gate is not a small detail. It is the stage that decides whether the attitude survives the burn at all.

9. The complementary filter

A complementary filter runs beside the Kalman filter on the same inputs and with the same trust gate. It isn't used to fly the rocket. It is there so the extra complexity of the Kalman filter can be judged against something simpler.

$$\alpha = \frac{\tau}{\tau + \Delta t}$$

$$\theta = \alpha(\theta + \omega \Delta t) + (1 - \alpha)\theta_{acc}$$

The time constant τ is set to 0.5 seconds. What this does is take the fast part of the gyro signal and the slow part of the accelerometer signal and add them together, which is more or less the same idea as the Kalman filter but with a fixed weighting instead of one that is worked out from how uncertain each state currently is.

10. Tilt compensated heading

The heading in the original firmware was worked out straight from two magnetometer axes. That form has two problems at once.

The first is that it assumes the board is lying flat. Once the board is tilted, part of the vertical component of the magnetic field leaks into the horizontal axes and the heading swings by tens of degrees even though the rocket never turned. In order to fix this the field is rotated back to level first, using the roll and the pitch that the Kalman filter already produced:

$$X_h = m_x \cos\ \theta\ + m_z \sin\ \theta$$

$$Y_h = m_x \sin\ \varphi\ \sin\ \theta\ + m_y \cos\ \varphi\ - m_z \sin\ \varphi\ \cos\ \theta$$

$$heading\ =\ atan2\left(Y_h, X_h\right)$$

The second problem is quieter and it is easier to miss. The magnetometer inside the ICM20948 is a separate part and it doesn't share its axes with the accelerometer and the gyroscope. Its X is the accelerometer Y, its Y is the accelerometer X, and its Z points the other way. The original code never undid that swap, so the heading was wrong by a fixed amount before the tilt was even considered. The swap is undone inside the heading calculation only, so the logged magnetometer channels stay directly comparable between the raw and the filtered version.

11. How it fits into the firmware

The chain lives in Filters.h and Filters.cpp. It is started once from setup, and after that it is called from readIMU the moment a new sample lands. Being called there and not from the main loop is deliberate, because it means the Δt used by the low pass and by the Kalman filter is the real interval between two sensor samples rather than however long the loop happened to take.

Every filtered channel is written to the SD card next to its raw twin, and the log line carries 54 columns in total. The flight state machine reads the filtered values for the launch and apogee tests. On the console the K key starts a fresh gyro calibration and the R key switches the radio between sending the raw values and the filtered ones, although the card keeps recording both either way.

The whole chain costs about 2.3 kB of program memory, which was found by building the firmware twice with the switch in Config.h turned on and then off. The complete sketch uses 33 percent of the available program space and 7 percent of the memory, so there is plenty of room left.

On top of that, the filter states are reset if the sensor drops out and comes back, but the measured bias is kept. There is no chance to measure it again in the middle of a flight, so throwing it away would make things worse.

There is also a switch that compiles the chain out completely. When it is off, the filtered columns simply mirror the raw ones, and that gives a clean way to compare the two without changing anything else in the build.

12. Results and discussion

The figures below were produced by replaying a flight through the same C++ that runs on the board. The host tool links Filters.cpp directly, so if a cutoff or a gain is changed in Config.h the results move with it and there is no second copy of the maths that could quietly disagree.

Figure 1 — Axial acceleration before and after filtering. Top: the whole flight. Bottom: a zoom on the 1.7 s motor burn.

Figure 1 shows what the low pass actually does. The upper plot covers the whole flight and the lower one zooms into the burn, which is where the vibration lives. During the burn the raw trace swings by roughly ±20 m/s² around the real thrust curve while the filtered trace follows the curve itself. Away from the burn the two are almost on top of each other, and that is the point. A filter that changed the signal when there was nothing to remove would be doing damage.

Figure 2 — Pitch rate before and after filtering. Bottom: the pad, showing the zero-rate bias and the moment calibration finishes.

The lower plot of Figure 2 is the more interesting one. The rocket is sitting still, so the rate should read zero, but the raw trace sits noticeably below the line at about −1.3 deg/s. That offset is the bias, and integrating it is where the drift comes from. As can be noticed from the figure above, there is a step where the filtered trace jumps onto zero. That step is the moment the calibration on the pad finished and the measured bias started being subtracted.

The calibration recovered 0.835, −1.288 and 0.429 deg/s on the three axes, against the values of 0.85, −1.30 and 0.42 that were put into the test data. So the routine gets the bias back to within about 0.02 deg/s, which is close enough that whatever is left will take minutes rather than seconds to matter.

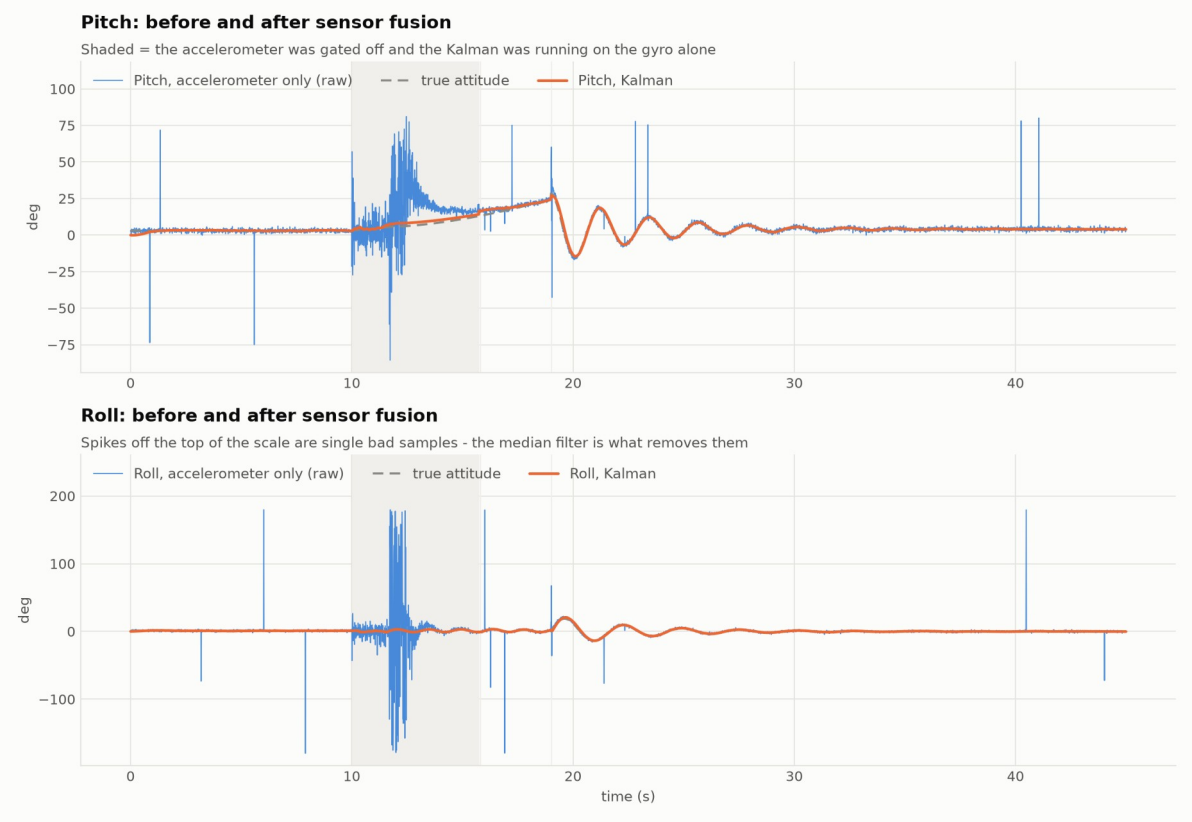

Figure 3 — Pitch and roll: accelerometer alone vs the Kalman filter, against the true attitude. Shaded = accelerometer gated off.

Figure 3 is the main result. The thin trace is the angle worked out from the accelerometer on its own, the thick one is the Kalman output and the dashed one is the true attitude. The shaded band is where the trust gate had switched the accelerometer off.

Inside that band the accelerometer angle runs away to nearly ±90 degrees of nonsense while the Kalman output stays on the truth. This is worth being clear about. That error isn't noise and it isn't something a better low pass would have caught. The sensor was measuring the thrust of the motor and reporting it as if it were gravity, so the number was wrong for a physical reason and the only way out of it was to stop believing the sensor for a while.

That is the whole reason the gate exists.

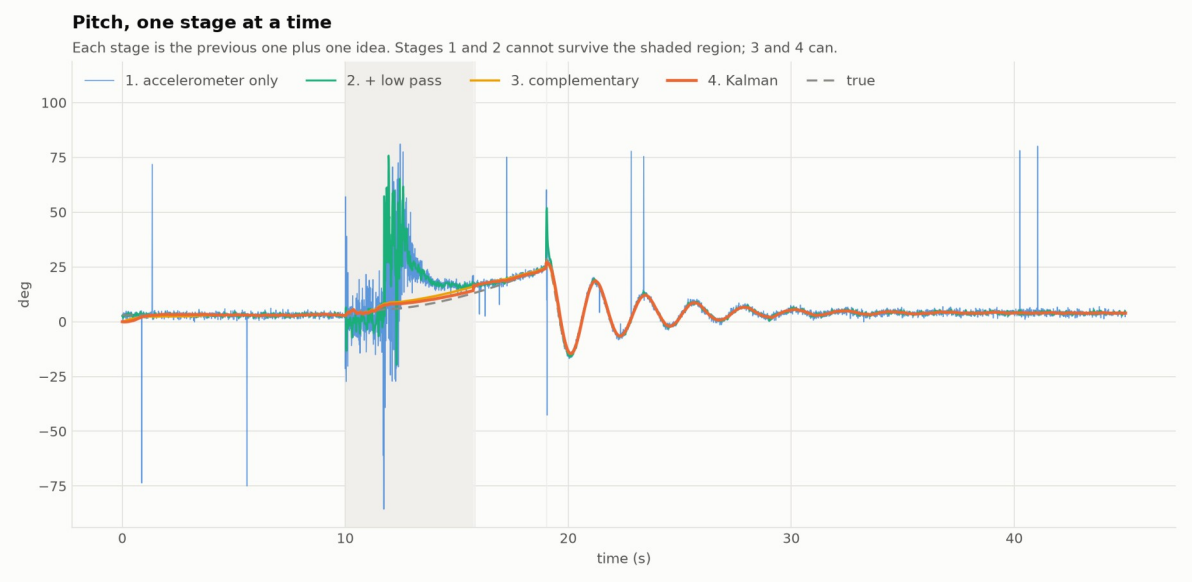

Figure 4 — Pitch through all four stages, showing what each one adds.

Figure 4 puts all four stages on one axis. Each one is the previous stage plus one idea. What it shows is that stages one and two both fail inside the shaded region while stages three and four survive it, and that difference is the whole argument for using sensor fusion instead of just smoothing harder.

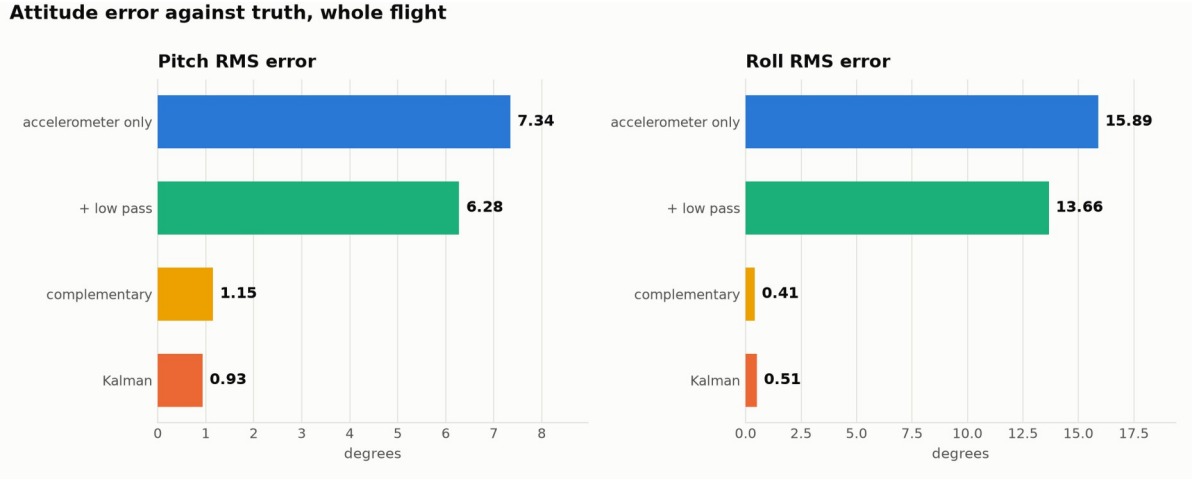

Figure 5 — RMS attitude error for each method, measured against the true attitude.

Figure 5 turns the same thing into a number. Pitch error drops from 7.34 degrees down to 0.93 degrees and roll error drops from 15.89 degrees to 0.51 degrees. The low pass on its own only takes pitch from 7.34 to 6.28, which is a reduction of about 14 percent, and that is small because the error it is fighting is mostly not noise.

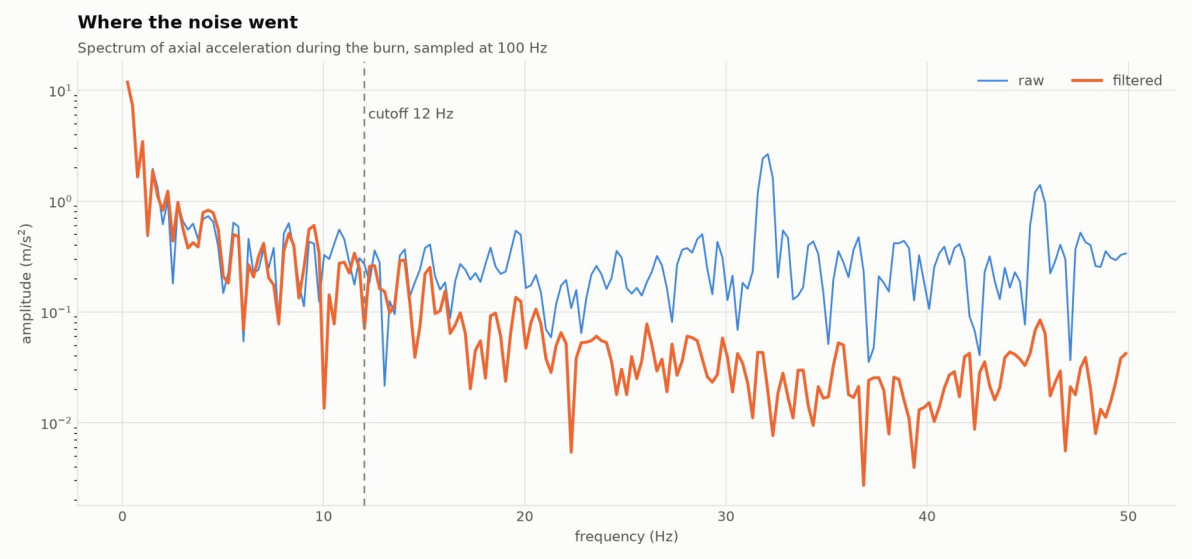

Figure 6 — Spectrum of the axial acceleration during the burn, before and after filtering.

Figure 6 shows where the noise went. The vibration of the airframe shows up as a clear peak at around 32 Hz in the raw trace and it is knocked down by roughly ten times in the filtered one. Below the cutoff the two traces sit on top of each other, which is the thing that actually has to be proven. Removing the vibration is only useful if the flight dynamics are still there afterwards.

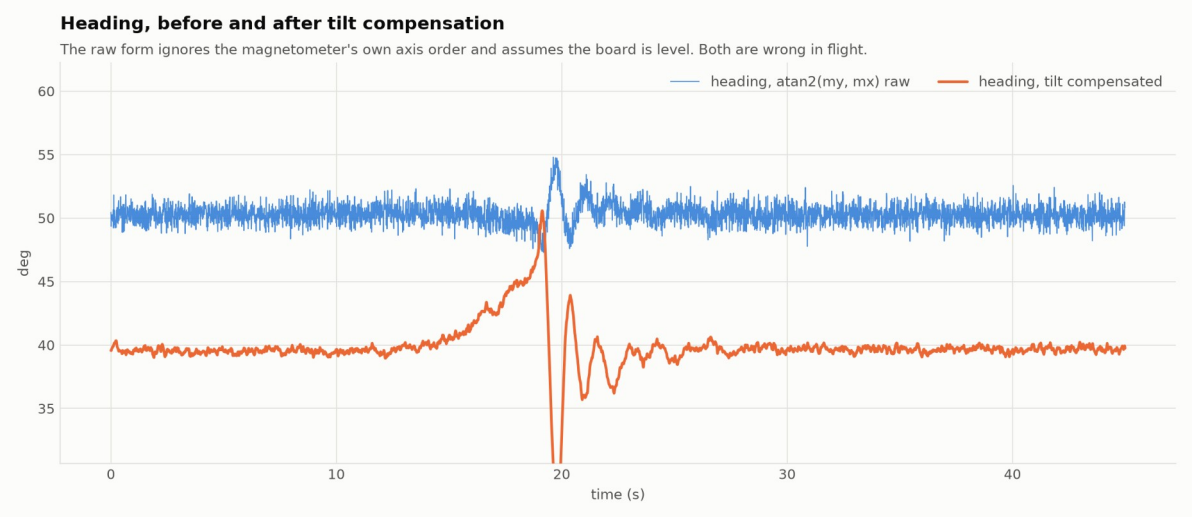

Figure 7 — Heading, raw atan2 vs tilt compensated.

Figure 7 shows the heading before and after. There is a gap of about 10 degrees between the two, and that gap is the two problems described earlier appearing together.

12.1 Measured noise reduction

Table 2 gives the noise on each channel while the rocket was still on the pad. The window starts after the calibration has finished, because measuring across that step would make the filtered trace sit at two different levels and the spread would come out wrong.

The spread is given as a median based estimate rather than the ordinary standard deviation. The reason is that the ordinary one gets dominated by whichever channel happened to catch a spike, so it ends up measuring luck instead of the filter. The median based version ignores the odd sample:

$$\sigma = 1.4826 \times median(|x - median(x)|)$$

Channel	Unit	σ raw	σ filtered	Reduction
Accel X	m/s²	0.174	0.090	48%
Accel Y	m/s²	0.156	0.079	49%
Accel Z	m/s²	0.148	0.076	49%
Gyro X	deg/s	0.566	0.293	48%
Gyro Y	deg/s	0.574	0.320	44%
Gyro Z	deg/s	0.579	0.291	50%

Table 2 — broadband noise on the pad (still, so every wiggle is noise). Spikes are counted separately.

All six channels land between 44 and 50 percent, and that agrees with what the theory says it should be. For a first order low pass with a smoothing factor of α acting on white noise the spread comes down by the square root of α divided by 2 minus α. At the 12 Hz accelerometer cutoff and a 100 Hz sample rate α works out at 0.43, which predicts a 48 percent reduction. The measured accelerometer channels came out at 48, 49 and 49 percent. So the filter is behaving the way a first order low pass is supposed to behave, and nothing odd is going on.

The spikes are handled separately because they belong to a different stage. Four of them were present in that window and all four were removed by the median.

12.2 Attitude error

Table 3 gives the error of each method against the true attitude, taken over the whole flight. The value quoted is the root mean square error, which is worked out by squaring the difference at every sample, taking the mean of those and then the square root.

Method	Pitch RMS error (deg)	Roll RMS error (deg)
Accelerometer only	7.34	15.89
Accelerometer and low pass	6.28	13.66
Complementary filter	1.15	0.41
Kalman filter	0.93	0.51

Table 3 — attitude error against truth, whole flight.

Two things in this table are worth pointing out. The first is the jump between the second and the third row. Going from raw to low passed only buys about 14 percent, but going

from low passed to a filter that also uses the gyro takes the pitch error down by more than five times. This supports the idea that the main problem was never the noise.

The second is that the complementary filter actually beats the Kalman filter on roll, coming in at 0.41 degrees against 0.51. It loses on pitch, 1.15 against 0.93, and it loses by more during the part of the flight where the accelerometer is gated off. So the Kalman filter is the better choice overall for a rocket, but on a mostly gentle flight the simpler filter is competitive and it would be dishonest to present it otherwise.

13. Limitations

The numbers above come from a modelled flight rather than a real one. This is on purpose, since the error against truth can only be worked out when the truth is known, and a real flight can't provide that. Everything is still produced by the real firmware code, and the noise model that was used is written down in the tool that generates the data. Once a real log exists, the same script reads it and most of the figures regenerate from real data. Figure 5 will drop out because there is no truth to compare against.

The mounting orientation matters and it isn't optional. Roll and pitch are taken about the X and Y axes with Z as the reference, so the board has to be mounted with its Z axis pointing up through the nose. If X is put along the airframe instead, the rocket sits at a pitch of −90 degrees while it is still on the pad, and that is exactly the point where the formula breaks down.

The card is written at 10 Hz while the sensor runs at about 100 Hz. The filtered columns are fine because they were worked out at the full rate and only sampled at 10 Hz for the log. The raw columns are decimated without any protection, so on a real log the vibration folds down to a lower frequency and the spectrum in Figure 6 can't be reproduced from the card. Dropping the log interval to 20 ms for one test flight would fix that.

The heading is still not calibrated for the magnetic distortion caused by the metal and the electronics around the sensor, so it can be treated as a relative bearing but not as a surveyed one.

14. Conclusion

The filter chain does two different jobs and it is worth keeping them apart. The low pass and the median together take the broadband noise down by about half on every channel and remove the spikes completely, and that part is straightforward. The bigger result is the second job. By tracking the gyro bias as a state and by refusing to believe the accelerometer while the motor is burning, the attitude error over the whole flight comes down from 7.34 degrees to 0.93 degrees in pitch and from 15.89 degrees to 0.51 degrees in roll.

A low pass filter on its own could never have done that, and Figure 4 shows why. Most of the error was not noise sitting on a good signal. It was a sensor reporting a physically

wrong answer for a few seconds, and the fix was to stop trusting it and to carry on using something else.